TableReser
TECHNICAL DOCUMENTATION
V1.0

OVERVIEW
- Project Overview
- Technology Stack
- User Roles

ARCHITECTURE
- System Architecture
- Component Architecture
- Data Models
- State Management

DATA FLOW DIAGRAMS
- Context Diagram (L0)
- L1 — System Processes
- L2 — Reservation Flow
- Authentication Flow

API REFERENCE
- Auth Endpoints
- Restaurants
- Reservations
- Reviews
- Users & Admin

FRONTEND
- Page Routes
- Custom Hooks

OPERATIONS
- Security
- Deployment
- Environment Variables

• v1.0 · Production Documentation

TableReser
Platform Docs

Full-stack restaurant reservation platform. Complete technical reference covering system architecture, data flow diagrams, API specification, and deployment guide.

`Stack: MERN` `Auth: JWT / bcrypt` `Storage: Cloudinary` `DB: MongoDB Atlas` `Frontend: React 18 + Redux Toolkit` `Build: Vite`

01 Project Overview

TableReser is a production-grade web application that enables diners to discover restaurants and make real-time table reservations while giving restaurant owners a complete management dashboard. A super-admin layer provides platform-wide governance — approving new restaurant applications, managing users, and overseeing all reservations.

PROBLEM SOLVED
Eliminates phone-tag between diners and restaurants by providing an always-on, self-service reservation channel with real-time availability.

KEY DIFFERENTIATORS
Three-tier role system, restaurant on-boarding pipeline with admin approval, image hosting via Cloudinary, and owner review-response capability.

PRIMARY USERS
Diners seeking hassle-free bookings, restaurant owners managing daily flow, and platform administrators governing quality.

SCALABILITY
Stateless JWT auth, MongoDB Atlas cloud DB, and Cloudinary CDN allow horizontal scaling of the API tier independently.

02 Technology Stack

Frontend

- ⚛ **React 18.3** — Component UI library with hooks
- 🗃 **Redux Toolkit 2.x** — Global state + async thunks
- 🛤 **React Router DOM 7** — Client-side routing & protected routes
- 🌐 **Axios 1.7** — HTTP client with interceptors
- 🎨 **Tailwind CSS 3.4** — Utility-first styling
- 📊 **Recharts 3** — Data visualisation in dashboards
- 🔔 **React Hot Toast** — Non-blocking user notifications
- ⚡ **Vite 6** — Build tool & dev server

Backend

- 🟩 **Node.js + Express 4** — REST API server, ES modules
- 🍃 **MongoDB + Mongoose 8** — Document DB with schema ODM
- 🔐 **JWT (jsonwebtoken 9)** — Stateless authentication, 7-day expiry
- 🔑 **bcryptjs** — Password hashing (12 salt rounds)
- ☁ **Cloudinary + multer** — Image upload, CDN delivery
- 📦 **dotenv + cors** — Config management, cross-origin

03 User Roles & Permissions

The platform enforces a strict three-tier role hierarchy. Roles are stored on the User document and enforced by the `authorize()` middleware on every protected route.

👑 **Super Admin**
- View & manage all users
- Activate / deactivate accounts
- Review restaurant applications
- Approve / reject applications
- Access all reservations
- Create restaurants directly
- Full system oversight

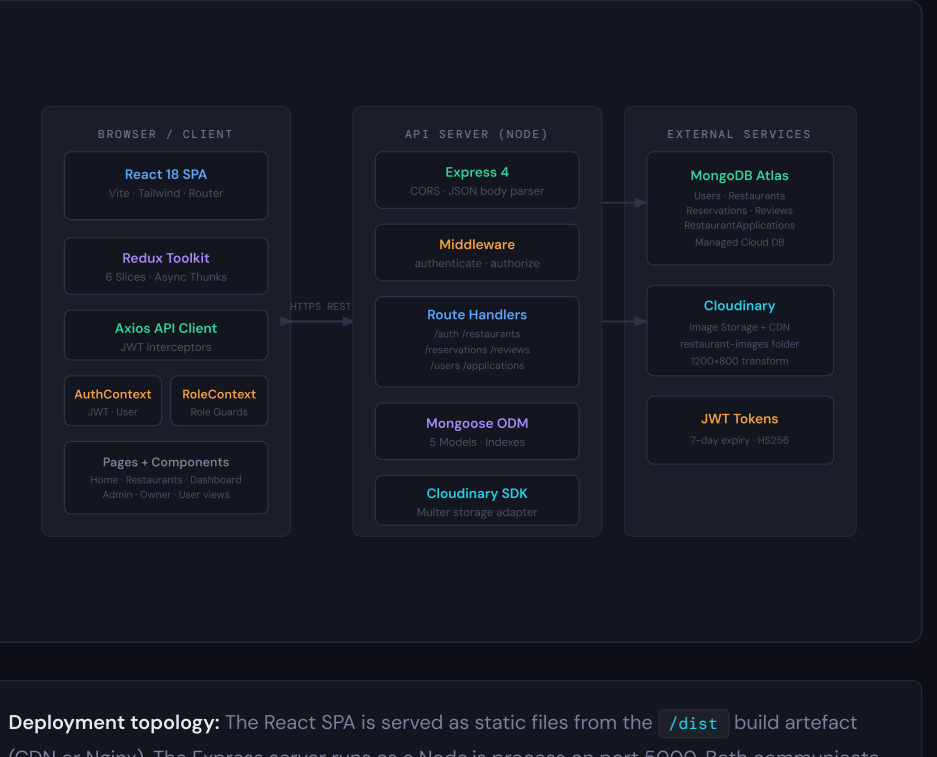
🍽 **Restaurant Owner**
- Submit restaurant application
- Manage own restaurant profile
- Upload restaurant images
- Configure menu & tables
- Set operating hours
- View incoming reservations
- Respond to guest reviews

👤 **User (Diner)**
- Browse all restaurants
- View menus & availability
- Make reservations
- Cancel reservations
- Leave reviews & ratings
- Apply to become owner
- Manage profile

RESOURCE	PUBLIC	USER	OWNER	SUPER ADMIN
`/api/restaurants (GET)`	✅	✅	✅	✅
`/api/restaurants (POST)`	❌	❌	✅	✅
`/api/restaurants/:id (PUT)`	❌	❌	Own only	✅
`/api/reservations (GET)`	❌	Own only	Own rest.	✅ All
`/api/reservations (POST)`	❌	✅	✅	✅
`/api/applications (GET)`	❌	Own only	Own only	✅ All
`/api/users (GET)`	❌	Self only	Self only	✅ All
`/api/reviews (POST)`	❌	✅	✅	✅
`/api/reviews (respond)`	❌	❌	Own rest.	✅

04 System Architecture

TableReser follows a classic client-server architecture with a clear separation between the React SPA (Single Page Application) and the Express REST API. External services (MongoDB Atlas, Cloudinary) are consumed exclusively by the backend.



Deployment topology: The React SPA is served as static files from the `/dist` build artefact (CDN or Nginx). The Express server runs as a Node.js process on port 5000. Both communicate over HTTPS — the frontend sends a Bearer JWT token on every authenticated request.

05 Frontend Component Architecture

The frontend uses a feature-based structure under `src/`. Each concern has a dedicated directory. Role-specific views are co-located under a sub-folder matching the role name.

```
src/
├── App.jsx                # Root router + layout shell
├── main.jsx               # ReactDOM entry, Redux Provider
├── index.css              # Global Tailwind base
│
├── pages/                 # Route-level page components
│   ├── Home.jsx
│   ├── Login.jsx  Register.jsx
│   ├── Restaurants.jsx  RestaurantDetail.jsx
│   ├── MyReservations.jsx
│   ├── Dashboard.jsx      # Role redirect dispatcher
│   ├── ApplyRestaurant.jsx
│   ├── ManageRestaurant.jsx
│   ├── ManageReviews.jsx
│   ├── AdminUsers.jsx
│   ├── AdminRestaurantApplications.jsx
│   ├── PrivacyPolicy.jsx  TermsOfService.jsx
│   ├── admin/Dashboard.jsx
│   ├── restaurant-owner/Dashboard.jsx
│   └── user/Dashboard.jsx
│
├── components/
│   ├── Navbar.jsx  Footer.jsx  ProtectedRoute.jsx
│   ├── admin/DashboardCard.jsx
│   ├── restaurant-owner/DashboardCard.jsx
│   ├── user/DashboardCard.jsx
│   └── common/
│       ├── Card.jsx  EmptyState.jsx  ErrorMessage.jsx
│       ├── LoadingSpinner.jsx  Pagination.jsx
│       └── SuccessMessage.jsx
│
├── context/
│   ├── AuthContext.jsx    # isAuthenticated, user, login/logout
│   └── RoleContext.jsx    # role-based UI helpers
│
├── hooks/                 # Data-fetching hooks wrapping Redux thunks
│   ├── useAuth.js  useUser.js  useApplication.js
│   └── useReservation.js  useRestaurant.js  useReview.js
│
├── store/
│   ├── index.js           # configureStore
│   └── slices/
│       ├── authSlice.js  userSlice.js  applicationSlice.js
│       └── reservationSlice.js  restaurantSlice.js  reviewSlice.js
│
├── api/
│   └── index.js           # Axios instance, base URL, JWT header injection
│
└── utils/
    ├── constants.js       # API URLs, enums
    └── helpers.js         # Date formatting, status mappers
```

Data Flow: Component → Store → API

1. **User Interaction**
   User clicks "Reserve Table" on `RestaurantDetail.jsx`.
2. **Custom Hook**
   `useReservation()` hook is called, dispatching the `createReservation` async thunk from `reservationSlice`.
3. **Axios API Call**
   The thunk calls `api.post('/reservations', payload)`. The Axios instance automatically attaches the Bearer JWT from `localStorage`.
4. **Express + Mongoose**
   Backend validates auth, checks table availability, creates the Reservation document, and returns 201.
5. **Redux State Update**
   The fulfilled action updates `reservationSlice.reservations`. React re-renders with the new list.
6. **Toast Notification**
   `react-hot-toast` displays a success message to the user.

06 Data Models (MongoDB Schemas)

User

FIELD	TYPE	CONSTRAINTS	NOTES
`name`	String	required, trim	Display name
`email`	String	required, unique, lowercase	Login credential
`password`	String	required, minlength:6	Bcrypt hash (12 rounds)
`role`	Enum	super_admin | restaurant_owner | user	Default: user
`restaurantId`	ObjectId ref Restaurant	nullable	Set on owner approval
`isActive`	Boolean	default: true	Admin can deactivate
`createdAt / updatedAt`	Date	auto	Mongoose timestamps

Restaurant

FIELD	TYPE	NOTES
`name`	String	Restaurant display name
`description`	String	Markdown-safe text
`location`	Object	address, city, state, zipCode
`cuisine`	String	e.g. Italian, Indian
`images`	[String]	Cloudinary URLs
`menu`	[MenuItem]	name, description, price, category, isAvailable
`tables`	[TableSchema]	tableNumber, capacity, isAvailable
`operatingHours`	Object	Mon–Sun: open, close, isClosed
`ownerId`	ObjectId ref User	required
`isVerified`	Boolean	Set true by admin on approval
`isActive`	Boolean	Soft delete flag

Reservation

FIELD	TYPE	NOTES
`userId`	ObjectId ref User	Who made the booking
`restaurantId`	ObjectId ref Restaurant	Where
`date`	Date	Reservation date
`time`	String	HH:MM format
`partySize`	Number	min: 1
`tableNumber`	Number	Optional assigned table
`status`	Enum	pending | confirmed | cancelled | completed
`specialRequests`	String	Free text
`contactPhone`	String	Guest contact number

RESERVATION STATUS LIFECYCLE

🟠 `pending`	Initial state on creation; awaiting restaurant confirmation.
🟢 `confirmed`	Restaurant owner has confirmed the booking.
🔴 `cancelled`	Cancelled by user or owner before the visit.
🔵 `completed`	Visit occurred; enables review submission.

Review

FIELD	TYPE	NOTES
`restaurantId`	ObjectId	Indexed for fast lookup
`userId`	ObjectId	Reviewer reference
`rating`	Number	1–5 inclusive
`comment`	String	Trimmed text
`response`	Object	text, respondedAt, respondedBy (owner)
`images`	[String]	Cloudinary review photos
`visitDate`	Date	When the diner visited

RestaurantApplication

FIELD	TYPE	NOTES
`userId`	ObjectId	Applicant (user role)
`restaurantName, ownerName`	String	Business details
`phone, email, address, cuisine`	String	Contact & location info
`description`	String	Restaurant description
`licenseNumber`	String	Business licence
`businessProof`	String	File URL, nullable
`status`	Enum	pending | approved | rejected
`adminNotes`	String	Rejection reason or admin comment
`reviewedBy`	ObjectId ref User	Admin who processed it
`reviewedAt`	Date	Timestamp of review action

Compound Indexes: `restaurantApplications` has a compound index on `{userId, status}` to enforce one pending application per user and speed up admin queries. `reviews` indexes on `{restaurantId, createdAt}` for efficient paginated review loading.

07 State Management

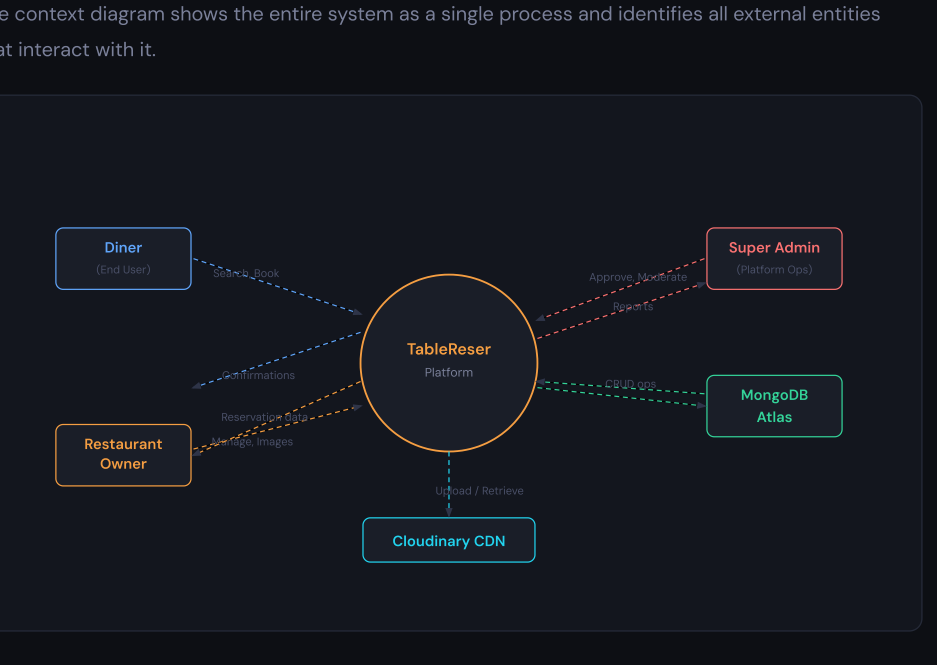
Global state is handled by Redux Toolkit. Six slices mirror the six backend resource domains. Each slice contains `createAsyncThunk` actions for every API call, and manages `loading`, `error`, and resource array state.

SLICE	KEY STATE	MAIN ACTIONS
`authSlice`	user, token, isAuthenticated	login, register, logout, getMe
`restaurantSlice`	restaurants[], currentRestaurant	fetchAll, fetchOne, create, update, delete, uploadImages
`reservationSlice`	reservations[], myReservations[]	fetchAll, create, updateStatus, cancel
`reviewSlice`	reviews[], restaurantReviews[]	fetchByRestaurant, create, respond, delete
`userSlice`	users[], currentUser	fetchAll, updateProfile, toggleActive
`applicationSlice`	applications[], myApplication	submit, fetchAll, approve, reject

Dual state pattern: `AuthContext` holds the raw JWT and minimal user object for fast route-guard decisions without touching Redux. Redux then holds the full enriched data objects for rendering. This avoids prop-drilling and keeps auth checks lean.

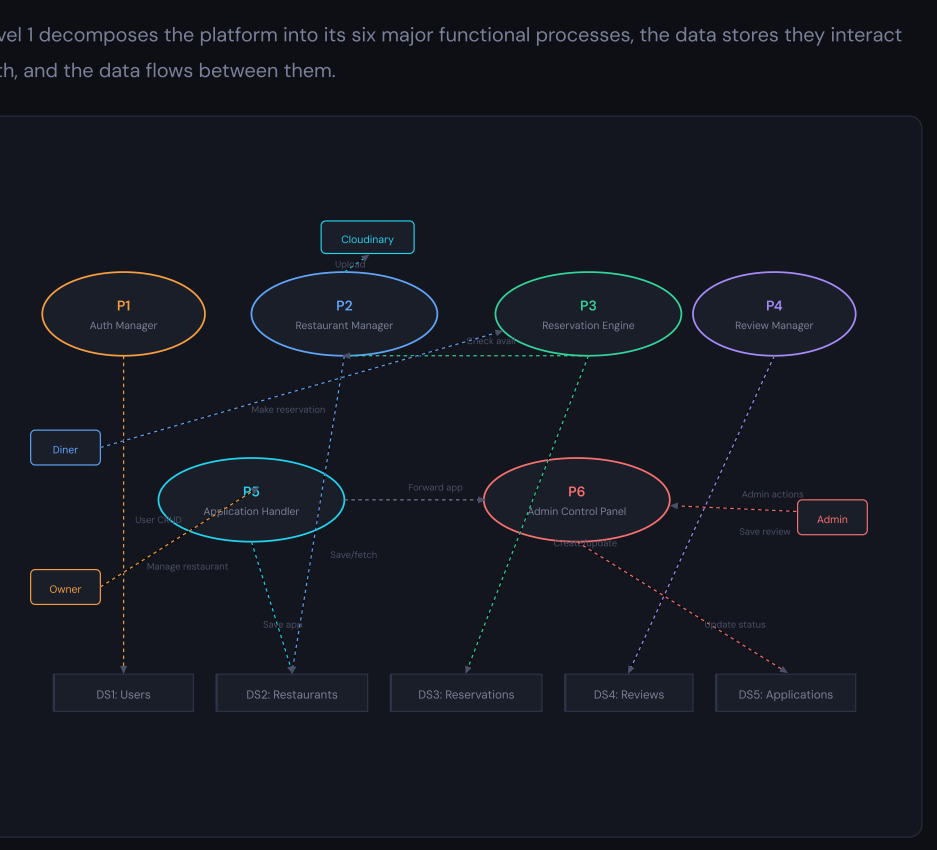
08 DFD Level 0 — Context Diagram

The context diagram shows the entire system as a single process and identifies all external entities that interact with it.



09 DFD Level 1 — System Processes

Level 1 decomposes the platform into its six major functional processes, the data stores they interact with, and the data flows between them.



P1 — AUTH MANAGER
Register, login, JWT issue, password hashing, profile fetch.

P2 — RESTAURANT MANAGER
CRUD operations on Restaurant docs, image upload to Cloudinary, menu & table management.

P3 — RESERVATION ENGINE
Table availability check, conflict detection, create/cancel/status-update reservations.

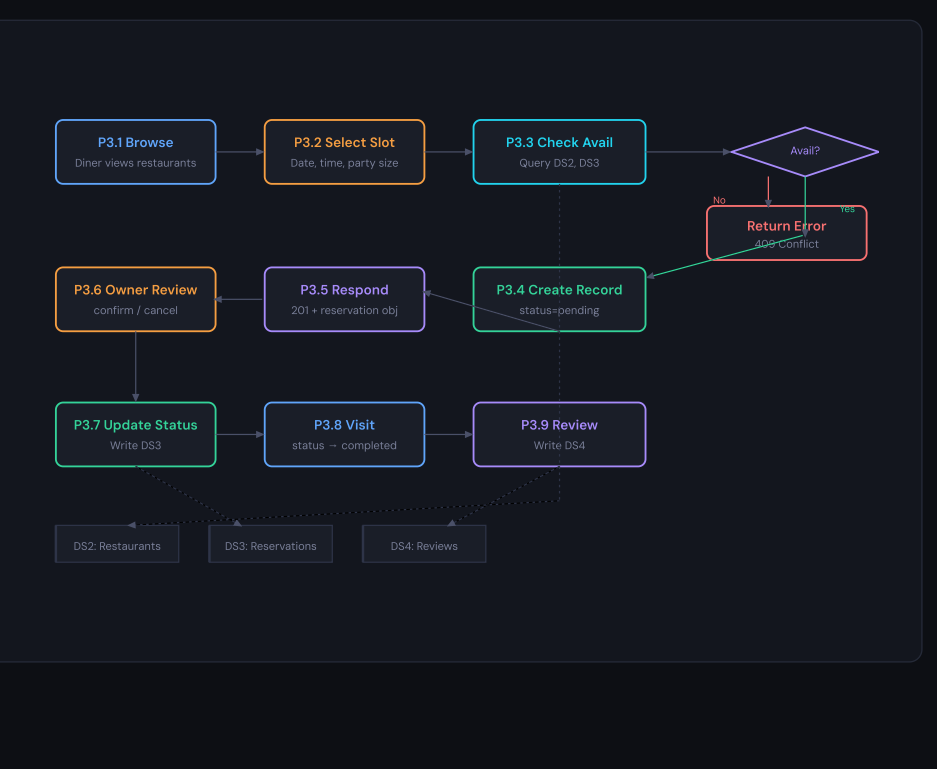
P4 — REVIEW MANAGER
Submit ratings & comments, owner responses, review listing by restaurant.

P5 — APPLICATION HANDLER
Restaurant on-boarding submissions, one-pending-per-user enforcement.

P6 — ADMIN CONTROL PANEL
User management, application approve/reject, platform-wide data access.

10 DFD Level 2 — Reservation Flow

This diagram zooms into Process P3, decomposing the complete end-to-end reservation lifecycle from discovery to completion.



11 Authentication Flow

1. **Registration — POST /api/auth/register**
   User submits `name`, `email`, `password`, optional `role`. Backend checks for duplicate email, creates User document with `bcrypt.hash(password, 12)`. Returns JWT (7-day) and user object.
2. **Login — POST /api/auth/login**
   Credentials validated against DB. `comparePassword()` runs `bcrypt.compare()`. Active check enforced. Returns JWT + user. Frontend stores token and populates `AuthContext`.
3. **Axios Interceptor — JWT Injection**
   Every Axios request has an interceptor that reads the token from state/localStorage and appends `Authorization: Bearer <token>` to the request headers automatically.
4. **authenticate() Middleware**
   On every protected route, `authenticate()` extracts the token, calls `jwt.verify()`, fetches the user from DB, and attaches `req.user` for downstream handlers.
5. **authorize(...roles) Middleware**
   Checks `req.user.role` against the allowed roles array. Returns `403 Forbidden` if the user lacks the required role. Used on every write endpoint.
6. **ProtectedRoute Component**
   On the frontend, `<ProtectedRoute />` reads `isAuthenticated` from `AuthContext`. If false, redirects to `/login`. Role-specific sub-routes redirect to the appropriate dashboard.

12 API Reference — Authentication

Base URL: `/api/auth`

POST	`/register`	Public	Register a new user
POST	`/login`	Public	Login and receive JWT
GET	`/me`	Auth	Get current user profile

```
// POST /api/auth/login — Request Body
{
  "email": "user@example.com",
  "password": "secure_pass"
}

// 200 Response
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "user": {
    "id": "64a...",
    "name": "Jane Doe",
    "email": "user@example.com",
    "role": "user",
    "restaurantId": null
  }
}
```

13 API Reference — Restaurants

Base URL: `/api/restaurants`

GET	`/ ?city=&cuisine=&search=`	Public	List all active restaurants
GET	`/:id`	Public	Get single restaurant detail
POST	`/`	Owner/Admin	Create restaurant
PUT	`/:id`	Owner/Admin	Update restaurant
POST	`/:id/images`	Owner/Admin	Upload images (multipart)
DELETE	`/:id/images/:imgId`	Owner/Admin	Remove a restaurant image
DELETE	`/:id`	Admin	Soft delete (isActive=false)

📸 Image uploads go through **multer → CloudinaryStorage**. The backend stores the returned Cloudinary HTTPS URLs in the `images[]` array. Images are resized to max 1200×800 on upload via Cloudinary transform params.

14 API Reference — Reservations

Base URL: `/api/reservations` — All endpoints require authentication.

GET	`/ ?status=&date=&restaurantId=`	Auth	List (role-filtered)
GET	`/:id`	Auth	Get single reservation
POST	`/`	Auth	Create reservation
PUT	`/:id/status`	Owner/Admin	Update reservation status
DELETE	`/:id`	Auth	Cancel (own reservation)

```
// POST /api/reservations — Request Body
{
  "restaurantId": "64a...",
  "date": "2025-12-25",
  "time": "19:00",
  "partySize": 4,
  "specialRequests": "Window table preferred",
  "contactPhone": "+91 9876543210"
}
```

15 API Reference — Reviews

Base URL: `/api/reviews`

GET	`/restaurant/:restaurantId`	Public	Get reviews for a restaurant
POST	`/`	Auth	Submit a review (rating 1–5)
PUT	`/:id/respond`	Owner	Owner responds to review
DELETE	`/:id`	Admin	Remove inappropriate review

16 API Reference — Users & Admin

Base URL: `/api/users` and `/api/applications`

GET	`/api/users`	Admin	All platform users
PUT	`/api/users/:id/status`	Admin	Activate / deactivate user
PUT	`/api/users/:id`	Self	Update own profile
GET	`/api/applications`	Admin	All restaurant applications
POST	`/api/applications`	Auth	Submit restaurant application
PUT	`/api/applications/:id/approve`	Admin	Approve — creates Restaurant + updates User role
PUT	`/api/applications/:id/reject`	Admin	Reject with optional admin note

17 Frontend Page Routes

PATH	COMPONENT	ACCESS	PURPOSE
`/`	Home	Public	Landing page with featured restaurants
`/login`	Login	Public (guest only)	Email/password login form
`/register`	Register	Public (guest only)	New user registration
`/restaurants`	Restaurants	Public	Browse & filter all restaurants
`/restaurants/:id`	RestaurantDetail	Public	Detail + reservation form
`/privacy-policy`	PrivacyPolicy	Public	Legal privacy page
`/terms-of-service`	TermsOfService	Public	Terms & conditions
`/dashboard`	Dashboard	Auth	Role-based redirect dispatcher
`/admin/dashboard`	admin/Dashboard	Admin	Platform KPIs & quick actions
`/restaurant-owner/dashboard`	owner/Dashboard	Owner	Restaurant metrics & reservations
`/user/dashboard`	user/Dashboard	User	Personal reservation history
`/my-reservations`	MyReservations	Auth	Manage personal bookings
`/apply-restaurant`	ApplyRestaurant	Auth	Restaurant on-boarding form
`/manage-restaurant`	ManageRestaurant	Owner	Edit restaurant, menu, tables
`/manage-reviews`	ManageReviews	Owner	View & respond to reviews
`/admin/users`	AdminUsers	Admin	User list with activate/deactivate
`/admin/restaurant-applications`	AdminRestaurantApplications	Admin	Application review queue

18 Custom Hooks

Each resource domain has a corresponding hook that wraps Redux dispatch calls and exposes clean data and action functions to page components.

HOOK	RETURNS	KEY ACTIONS
`useAuth()`	user, isAuthenticated, loading	login(), register(), logout()
`useRestaurant()`	restaurants[], currentRestaurant, loading	fetchRestaurants(), createRestaurant(), uploadImages()
`useReservation()`	reservations[], loading, error	createReservation(), updateStatus(), cancelReservation()
`useReview()`	reviews[], loading	fetchReviews(), submitReview(), respondToReview()
`useUser()`	users[], currentUser	fetchUsers(), updateProfile(), toggleActive()
`useApplication()`	applications[], myApplication	submitApplication(), fetchApplications(), approve(), reject()

19 Security Architecture

AUTHENTICATION
JWT with 7-day expiry signed with HS256. Tokens stored client-side. Server re-validates the user record from DB on every request (protects against deactivated accounts bypassing auth).

PASSWORD SECURITY
bcryptjs with 12 salt rounds. Passwords never returned in API responses. Minimum 6-character policy enforced at schema level.

AUTHORIZATION
Every protected route double-checks both authentication (valid JWT) and authorization (correct role). Owners can only modify their own restaurant, enforced by comparing `ownerId` with `req.user._id`.

INPUT HANDLING
Express `json()` body parser, Mongoose schema validation, and `trim` flags on string fields. File uploads restricted to jpg/jpeg/png/gif/webp by multer config.

CORS
CORS middleware applied globally. For production, the origin whitelist should be restricted to the frontend domain via `cors({ origin: 'https://app.tablereser.com' })`.

SOFT DELETE
Restaurants and users use `isActive` boolean flags rather than hard deletes. This preserves referential integrity in historical reservations and reviews.

20 Deployment Guide

Frontend Build

```
# Install dependencies
cd Frontend
npm install

# Create .env
echo "VITE_API_URL=https://api.tablereser.com" > .env

# Production build
npm run build

# Output: /dist — deploy to Vercel / Netlify / Nginx
```

Backend Setup

```
# Install dependencies
cd Backend
npm install

# Set environment variables (see next section)
cp .env.example .env

# Start production server
npm start

# Or with PM2 process manager:
pm2 start src/index.js --name tablereser-api
```

Recommended hosting: Backend → Railway / Render / AWS EC2. Frontend → Vercel (auto-deploys from git). Database → MongoDB Atlas free/paid tier. Images → Cloudinary free tier (25GB storage).

21 Environment Variables

Backend — `.env`

VARIABLE	REQUIRED	DESCRIPTION
`PORT`	No (default 5000)	HTTP server port
`MONGOURI`	Yes	MongoDB connection string (Atlas)
`JWT_SECRET`	Yes	Strong random string for JWT signing
`CLOUDINARY_CLOUD_NAME`	Yes	Cloudinary account cloud name
`CLOUDINARY_API_KEY`	Yes	Cloudinary API key
`CLOUDINARY_API_SECRET`	Yes	Cloudinary API secret
`NODE_ENV`	No	Set to `production` for prod

Frontend — `.env`

VARIABLE	REQUIRED	DESCRIPTION
`VITE_API_URL`	Yes	Backend API base URL (no trailing slash)

⚠ Never commit `.env` files to version control. Use a secrets manager (AWS Secrets Manager, Railway environment variables, or Vercel env settings) for production deployments. Rotate the `JWT_SECRET` periodically — all existing tokens will be invalidated.

TableReser Platform

Documentation v1.0 · Generated February 2026 · MERN Stack · Production-Ready